

Concurrent Programming TDA384/DIT392

26 October 2024

Exam supervisors: N. Piterman (piterman@chalmers.se, 031 772 6053)
and G. Schneider (gersch@chalmers.se, 031 772 6073)

(Exam set by N. Piterman and G. Schneider, based on the course given in
Aug-Oct 2023 and Jan-Mar 2024)

Material permitted during the exam (hjälpmedel):

Two textbooks; four sheets of A4 paper with notes (single or double-sided);
English dictionary (no smart phones allowed).

Grading: You can score a maximum of 70 points. Exam grades are: between 28–41 (3), between 42–55 (4), 56 or more (5).

Passing the course requires passing the exam and passing the labs. The overall grade for the course is determined as follows: between 40–59 (3), between 60–79 (4), 80 or more (5).

The exam **results** will be available in Ladok within 15 *working* days after the exam's date.

Instructions and rules:

- Please write your answers clearly and legibly: unnecessarily complicated solutions will lose points, and answers that cannot be read will receive no points!
- Justify your answers, and clearly state any assumptions that your solutions may depend on for correctness.
- Answer each question on a new page. Glance through the whole paper first; five questions, numbered Q1 through Q5. Do not spend more time on any question or part than justified by the points it carries.
- Be precise. In your answers, try to use the programming notation and syntax used in the questions. You can also use pseudo-code, *provided* the meaning is precise and clear. If need be, explain your notation.

This page is left empty intentionally.

WITH ANSWERS!!!

Q1 (12p). In what follows you have 4 sub-questions (Parts a to d) concerning different topics seen in the course. Each part a multiple-choice question. The grading for each part is as follows:

- Correct: 3 points;
- No answer: 0 points;
- Incorrect: -1 point (negative).

The total number of points will be done by summing all the points for each part capped from below at 0. That is, minus points are *local* to this question but do not affect other questions.

We did not apply the -1 penalty for incorrect answers in this question.

NOTE: For each sub-question, there is exactly one option that is correct.

(Part a). (3p)

Consider the following program:

```
6      private static volatile boolean flag;
7
8      public void function1() {
9
10         final Thread t1 = new Thread(() -> {
11             for (int i=0 ; i<10 ; i++) {
12                 flag = true;
13             }
14         });
15
16         final Thread t2 = new Thread(() -> {
17             for (int i=0 ; i<10 ; i++) {
18                 flag = true;
19             }
20         });
21
22         t1.start();
23         t2.start();
24     }
```

- a) It has a data race.
- b) It does not have a data race but has a race condition.
- c) It has both a data race and a race condition.
- d) It is fine.

Answer: Option d). The only shared variable is `flag`. It is declared as `volatile`. Hence, the writes to the variable are synchronized and there

is no data race. There is no specification so we cannot know if there is a race condition.

(Part b). (3p)

Consider the following program:

```
34     private volatile int counter;
35
36     public void function2() {
37         final Thread t3 = new Thread() -> {
38             for (int i=0 ; i<10 ; i++) {
39                 int cnt = counter;
40                 counter = cnt + 1;
41             }
42         };
43
44         final Thread t4 = new Thread() -> {
45             for (int i=0 ; i<10 ; i++) {
46                 int cnt = counter;
47                 counter = cnt + 1;
48             }
49         };
50
51         t3.start();
52         t4.start();
53     }
```

- a) It has a data race.
- b) It does not have a data race but has a race condition.
- c) It has both a data race and a race condition.
- d) It is fine.

Answer: Option d). The only shared variable is counter. It is declared as volatile. Hence, the writes to the variable are synchronized and there is no data race. There is no specification so we cannot know if there is a race condition.

(Part c). (3p)

- a) In Erlang one cannot have race conditions.
- b) In Erlang one cannot have data races.
- c) In Erlang neither data races nor race conditions are possible.
- d) In Erlang both data races and race conditions are possible.

Answer: Option b). There are no shared variables in Erlang. In particular, one cannot have two accesses to the same memory location.

(Part d). (3p)

Consider the following usage of monitor:

```
24 |         l.lock();
25 |         try {
26 |             if (check_something) {
27 |                 cond.await();
28 |             }
29 |             // do something
30 |         }
31 |         finally {
32 |             l.unlock();
33 |         }
```

- a) This is a good example of using a condition in a monitor.
- b) This is a good example of using a condition in a monitor under signal-and-wait.
- c) This is a good example of using a condition in a monitor under signal-and-continue.
- d) This is not the right way to use a condition in a monitor.

Answer: Option b). Under signal and continue, the thread resuming after the await cannot be sure that the conditions corresponding to the signal are still holding as they are not ensured to continue right after the signal. Under signal and wait, dually, the thread resuming after the await knows that the thread that signaled just set up the conditions so that the condition is true.

Q2 (18p). In what follows you will get 4 assertions concerning topics taught in the course. For each assertion, you need to say whether it is correct (true) or not (false). You need to justify your answer in each case (an answer without a justification will not be granted full points).

(Part a). (5p) You are using the software transactional memory that was taught in class to manage a set of variables (see below). Every time you finish a transaction you send all the variables you checked out back and their version numbers are updated.

```
stm(Storage, {pull, Name}) ->
  case dict:is_key(Name, Storage) of
    true ->
      {reply, Storage,
       dict:fetch(Name, Storage)};
    false ->
      {reply, Storage, not-found}
  end;

stm(Storage, {push, Vars}) ->
  case try-push(Vars, Storage) of
    {success, NewStorage} ->
      {reply, NewStorage, success};
    fail ->
      {reply, Storage, fail}
  end.

try-push([], Storage) ->
  {success, Storage};
try-push([Var = #var{name = Name, version = Version} | Vars], Storage) ->
  case dict:find(Name, Storage) of
    {ok, #var{version = Version}} ->
      try-push(Vars,
               dict:store(Name, Var#var{version = Version + 1},
                           Storage));
    _ -> fail
  end.
```

A colleague tells you that this approach is inefficient as it updates the version numbers of variables that have not changed. They suggest to send back only the variables that were changed and that those that were not changed do not need to be pushed back. This is a bad suggestion.

Answer: True. This is a bad suggestion. They are right that changing the version number of variables that were not changed is inefficient. But the correctness of the changes depends on all the read variables. In particular, in order to conclude atomicity of the computation the STM needs to know which variables were read as part of the computation.

(Part b). (5p)

You are struggling to create a pair of processes in Erlang that exchange messages with each other. A colleague suggests that you could use the following approach. Use the code below to create several channels (say a2b and b2a). Then, create the processes that you want to talk to each other (say a and b) and give them the channels that were created in advance.

```

1  -module(channel).
2  -export([start/1,get_from_channel/1,send_through_channel/2]).
3
4  start(Channel) ->
5  |   Pid = spawn(fun channel_fun/0),
6  |   catch unregister(Channel),
7  |   register(Channel, Pid).
8
9  channel_fun() ->
10 |   receive
11 |   |   {get, From1, Ref1 } ->
12 |   |   |   receive
13 |   |   |   |   { put, From2, Ref2, Payload } ->
14 |   |   |   |   |   From1 ! { reply, Ref1, Payload },
15 |   |   |   |   |   From2 ! { ack, Ref2 }
16 |   |   |   end,
17 |   |   channel_fun
18 |   end.
19
20 send_through_channel(Channel, Data) ->
21 |   Ref = make_ref(),
22 |   Channel ! { put, self(), Ref, Data},
23 |   receive
24 |   |   { ack, ref } -> ok
25 |   end.
26
27 get_from_channel(Channel) ->
28 |   Ref = make_ref(),
29 |   Channel ! {get, self(), Ref},
30 |   receive
31 |   |   {reply, Ref, Payload} -> Payload
32 |   end.

```

You go ahead and use this code. Just to be on the safe side, you use several channels for each process (so a2b1, a2b2, a2b3 and so on). Your colleague is alarmed and tells you that it is no longer the case that if process a sends two messages to process b they arrive in the order that they were sent. Your colleague is wrong.

Answer: False. Your colleague is right. If messages from a to b are split so that they are sent through a2b1 and a2b2 then b will receive the messages from different processes and there is no longer a guarantee that they will be forwarded in the order that they were sent.

(Part c). (5)

Your colleague then supplies you with this example of how to create two processes that communicate with each other:

```
28  compute() ->
29      ok.
30
31  send_to_other(Other,Data) ->
32      Other ! { Data, self() }.
33
34  get_from_other(Other) ->
35      receive
36      | { Data, Other } -> Data
37  end.
38
39  connected_process(Other) ->
40      Data = compute(),
41      send_to_other(Other,Data),
42      NewData = get_from_other(Other),
43      connected_process(Other).
44
45  unconnected_process() ->
46      receive
47      | Pid -> connected_process(Pid)
48  end.
49
50  main() ->
51      First = spawn(fun unconnected_process/0),
52      Second = spawn(fun () -> connected_process(First) end),
53      First ! Second.
54
```

This indeed solves the problem (and is a better and simpler option).

Answer: False. This does not solve the problem. There is a race between First ! Second. in line 53 and First ! { Data, Second }. in line 32, when run by Second. In case that the second message arrives before the first, First will try to execute { Data,Second } ! { Data, First }. In this case, an exception happens.

In case that the first message arrives before the second, the coupling works as expected.

This race turned out to be very hard to identify. So we accepted answers stating that this is a good solution and gave a bonus to those identifying the problem.

(Part d). (3p)

Consider the following usage of lock and semaphore:

```
10      public void useSemaphore() throws InterruptedException {
11          l.lock();
12          if (s.availablePermits() > 0) {
13              s.acquire();
14          } else {
15              l.unlock();
16              s.acquire();
17          }
18      }
```

The way the lock and semaphore are used is dangerous and could lead to a deadlock.

Answer: True. In case that the number of available permits is positive when checking it at line 12 does not guarantee that the acquire will succeed. There is no guarantee that line 13 will execute right after line 12. A thread running this sequence would then be blocked waiting for the semaphore while holding the lock.

Q3 (11p). Here is an skeleton of an implementation of an algorithm implemented using fork/join parallelism and a workers pool. We are interested in the number of tasks that are going to be created by a run of this algorithm.

The algorithm starts from a set of elements (**pairs**). Then, for every element of the set (line 26), it creates a smaller set with the rest of the elements (lines 27-28), and then creates and forks a recursive task for each of them (lines 33-34). Before making a final computation in **summarize()** (line 43), the algorithms joins all the recursive tasks that were created (lines 40-42).

```

11  static class StreettSolverTask extends RecursiveTask<Integer> {
12      private Integer something_important;
13      private HashSet<Integer> pairs;
14
15      public StreettSolverTask(HashSet<Integer> pairs, Integer something_important) {
16          this.pairs = pairs;
17          this.something_important = something_important;
18      }
19
20      protected Integer compute() {
21          if (pairs.size() <= 1) {
22              return something_important;
23          }
24
25          List<StreettSolverTask> l = new ArrayList<StreettSolverTask>();
26          for (Integer pair: pairs) {
27              HashSet<Integer> other_pairs = new HashSet<Integer>(pairs);
28              other_pairs.remove(pair);
29
30              // Do some important computation with
31              // the value of something important
32
33              StreettSolverTask t = new StreettSolverTask(other_pairs,something_important);
34              t.fork();
35
36              l.add(t);
37          }
38
39          List<Integer> results = new ArrayList<Integer>();
40          for (StreettSolverTask t : l) {
41              results.add(t.join());
42          }
43          return summarize(results);
44      }
45
46  > private Integer summarize(List<Integer> list) {-
49  }

```

(Part a). (6p)

If the initial task is created with a set of size 4, how many recursive tasks are going to be created (including the initial one). Explain your answer. It is OK to give a calculation without a final number.

Answer: We have the initial recursive task that has k pairs. It creates k recursive tasks with $k - 1$ pairs each. Only when getting to $k = 1$ the creation stops. So overall, we have $1 + k + k(k - 1) + k(k - 1)(k - 2) + \dots + k!$. Or in a formula: In general, the number of recursive tasks

that are created is:

$$1 + \sum_{i=0}^{k-1} k!/i! = \sum_{i=0}^k k!/i!.$$

In the case of $k = 4$, this is: $1 + 4 + 12 + 24 = 41$.

(Part b). (4p)

Discuss the potential advantages and disadvantages of running this parallel implementation compared with a direct application of Threads rather than using a pool and compared with a sequential implementation.

Answer: Just using threads is simpler to implement. But in this case we will have $k!$ threads, which is a very large number. So threads will be competing with each other for resources adding time spent on swapping and out of memory.

Using a pool, swapping of threads will not happen. It still uses quite complex concepts for doing something quite simple.

At the end $k!$ computations need to be done. So, for large number of k , the difference between $k!$ and $k!/n$, where n is the number of cores, may not be so significant. There is also a memory overhead of creating $k!$ objects, which needs to be taken into account.

Q4 (15p). Here is an Erlang implementation of a monitor with a single condition:

```

1  -module(monitor).
2  -export([start/0,lock/1,unlock/1,wait/1,signal/1]).
3
4  start() ->
5      spawn(fun () -> loop([]) end).
6
7  loop(Blocked) ->
8      receive
9          { lock, Owner } ->
10             Owner ! { ok, self() },
11             loop(Owner,Blocked)
12      end.
13
14  loop(Owner, Blocked) ->
15      receive
16          { unlock, Owner } -> loop(Blocked);
17          { wait, Owner } -> loop([Owner | Blocked]);
18          { signal, Owner } ->
19              case Blocked of
20                  [ Pid | Tail ] -> Pid ! { continue, self() };
21                  Tail -> ok
22              end,
23              loop(Owner,Tail)
24      end.
25
26  lock(Monitor) ->
27      Monitor ! { lock, self() },
28      receive
29          { ok, Monitor } -> ok
30      end.
31
32  unlock(Monitor) ->
33      Monitor ! { unlock, self() }.
34
35  wait(Monitor) ->
36      Monitor ! { wait, self() },
37      receive
38          { continue, Monitor } -> ok
39      end,
40      lock(Monitor).
41
42  signal(Monitor) ->
43      Monitor ! { signal, self() }.

```

(Part a). (3p)

Is the signalling policy implemented by the monitor singal-and-wait or signal-and-continue?

Answer: The code implements signal and continue. When the monitor gets a signal message it “releases” one waiting thread. That waiting thread is going to compete with other threads trying to lock the monitor.
v **(Part b). (8p)**

Modify the implementation so that it implements the other signaling policy.

Answer: Here are two potential implementation:

```

1  -module(monitor_sw).
2  -export([start/0,lock/1,unlock/1,wait/1,signal/1]).
3
4  start() ->
5      spawn(fun () -> loop([]) end).
6
7
8  loop(Blocked) ->
9      receive
10         { lock, Owner } ->
11             Owner ! { ok, self() },
12             loop(Owner,Blocked,false)
13     end.
14
15 loop(Owner, Blocked, Recursion) ->
16     receive
17         { unlock, Owner } when Recursion -> ok;
18         { unlock, Owner } -> loop(Blocked);
19         { wait, Owner } -> loop([Owner | Blocked]);
20         { signal, Owner } ->
21             case Blocked of
22                 [ Pid | Tail ] ->
23                     Pid ! { continue, self() },
24                     loop(Pid,Blocked,true);
25                 Tail -> ok
26             end,
27         Owner ! {continue, self() },
28         loop(Owner,Tail, Recursion)
29     end.
30
31 lock(Monitor) ->
32     Monitor ! { lock, self() },
33     receive
34         { ok, Monitor } -> ok
35     end.
36
37 unlock(Monitor) ->
38     Monitor ! { unlock, self() }.
39
40 wait(Monitor) ->
41     Monitor ! { wait, self() },
42     receive
43         { continue, Monitor } -> ok
44     end.
45
46 signal(Monitor) -> 14
47     Monitor ! { signal, self() },
48     receive
49         { continue, Monitor } -> ok
50     end.

```

```

1  -module(monitor_sw1).
2  -export([start/0,lock/1,unlock/1,wait/1,signal/1]).
3
4  start() ->
5      spawn(fun () -> loop([]) end).
6
7
8  loop(Blocked) ->
9      receive
10         { lock, Owner } ->
11             Owner ! { ok, self() },
12             loop(Owner,[], Blocked)
13     end.
14
15  loop(Owner, Signaled, Blocked) ->
16      receive
17         { unlock, Owner } ->
18             case Signaled of
19                 [ SHead | STail ] ->
20                     SHead ! {continue, self() },
21                     loop(SHead, STail, Blocked);
22                 _ -> loop(Blocked)
23             end;
24         { wait, Owner } ->
25             case Signaled of
26                 [ SHead | STail ] ->
27                     SHead ! { continue, self() },
28                     loop(SHead, STail, [Owner | Blocked]);
29                 _ -> loop([Owner | Blocked])
30             end;
31         { signal, Owner } ->
32             case Blocked of
33                 [ Pid | Tail ] ->
34                     Pid ! { continue, self() },
35                     loop(Pid, [Owner | Signaled], Tail);
36                 Tail ->
37                     Owner ! { continue, self() },
38                     loop(Owner, Signaled, Tail)
39             end
40     end.
41
42  lock(Monitor) ->
43      Monitor ! { lock, self() },
44      receive
45          { ok, Monitor } -> ok
46      end.
47
48  unlock(Monitor) ->
49      Monitor ! { unlock, self() }.
50
51  wait(Monitor) ->
52      Monitor ! { wait, self() },
53      receive
54          { continue, Monitor } -> ok
55      end.
56
57  signal(Monitor) ->
58      Monitor ! { signal, self() },
59      receive
60          { continue, Monitor } -> ok

```

(Part c). (4p)

Going back to the original implementation, sketch how you would modify it to have multiple conditions. There is no need to implement your solution but only explain how you would go about it.

You do not have to solve part b to score points in part c.

Answer: First, the messages handled by the loop would change to {signal, cond_name, owner} and {wait, cond_name, owner}.

*The state, instead of the list of **Blocked** would become **ConditionQueuesList**, where each condition queue is a tuple with the condition's name and the list of PID's waiting for this condition.*

*Then, upon getting a signal for a certain condition, search the **ConditionQueuesList** for this condition, remove one PID from its list and send it a wake-up message. If the condition is not found, then just ignore the signal message.*

*Upon getting a wait for a certain condition, search for this condition in **ConditionQueuesList**, if found, add the PID to that list, if not found, create a new pair with the condition name and a list containing the single PID.*

*The functions **wait** and **signal** would have to include an extra parameter with the name of the condition.*

Q5 (14p). Here are programs for controlling crossings of a bridge between the two peaks of the Kilimanjaro. The bridge cannot hold more than two people together. The programs try to be fair by not allowing two people to cross in one direction if someone is waiting to cross in the other direction. For each solution, identify whether the solution works as promised. If it works, explain why. If it does not work, show its problems.

The rest of this page is empty

(Part a). (7p) Here is a monitor based solution (note the use of monitor keyword).

```

5  monitor class Kilimanjaro1 {
6
7      int waitL=0,waitR=0,crossingL=0,crossingR=0;
8      Condition goL,goR;
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Answer: This implementation satisfies that at most two people cross the bridge simultaneously. Furthermore, if someone is waiting to cross

from Left and someone is already crossing from the right, then the person waiting to cross from the left will enter before other people from the right. And similarly for the other direction. However, there is a subtle scenario that could lead to starvation. A bonus was given to those who identified this problem.

The functions are completely symmetric, so the explanation is going to refer mostly to crossing from left to right.

First, we notice that as this is a monitor, methods are run in mutual exclusion and that when conditions are awaited, the control of the monitor is freed. It follows that there are no problems with data races or similar and the updates that are happening to variables are indeed doing what is expected. So, the number `wait` reflects those that have called `register` but have not finished `enter`. The number `crossing` reflects how many are crossing.

In `enterL`, if there are two currently crossing, or someone crossing from left and someone waiting on the right, the person waits for the condition that will allow to `goL`. Upon, finishing the wait, the condition is checked again, so it does not matter if the signalling policy is signal-and-wait or signal-and-continue.

Upon exiting, there is a preference to signal to people crossing from the other side. This requires a further check and, with the current implementation, could lead to starvation. Suppose that someone finishes crossing from the left but that this happens while someone is crossing from the right. Then, our left crosses finishes and signals `goR`. But `enterR` cannot complete. Because those entering from the right will not go in as long as someone is waiting on the left. So the signal sent by `exitL` is lost. Once the person crossing from the right is finished, if no one is waiting on the left, it will release the person waiting on the right. But if someone is waiting on the left, they will be released by a signal. Now, a fresh person comes from the right, they can go on the bridge. We are back at the scenario we started from and this can go on forever leading to starvation of those waiting.

(Part b). (7p) Here is a semaphore based solution.

```

3 public class Kilimanjaro2 {
4     Semaphore waitL;
5     Semaphore waitR;
6     Semaphore crossingL;
7     Semaphore crossingR;
8
9     void Kilomanjaro2() {
10         waitL = new Semaphore(permits:0);
11         waitR = new Semaphore(permits:0);
12         crossingL = new Semaphore(permits:1);
13         crossingR = new Semaphore(permits:1);
14     }
15
16     public void registerL() {
17         waitL.release();
18     }
19
20     public void registerR() {
21         waitR.release();
22     }
23
24     public void enterL() throws InterruptedException {
25         if (!crossingL.tryAcquire()) {
26             if (waitR.tryAcquire()) {
27                 // someone waiting on the other side
28                 waitR.release();
29                 crossingL.acquire();
30             } else {
31                 crossingR.acquire();
32             }
33         }
34         // Acquired an can cross the bridge
35         waitL.acquire();
36     }
37
38     public void enterR() throws InterruptedException {
39         if (!crossingR.tryAcquire()) {
40             if (waitL.tryAcquire()) {
41                 // someone waiting on the other side
42                 waitL.release();
43                 crossingR.acquire();
44             } else {
45                 crossingL.acquire();
46             }
47         }
48         // Acquired an can cross the bridge
49         waitR.acquire();
50     }
51
52     public void exitR() {
53         crossingR.release();
54     }
55
56     public void exitL() {
57         crossingL.release();
58     }
59 }

```

Answer: This solution does not work.

It is possible for someone entering from the left, to take the ticket from crossingR. This happens if no one is waiting on the right. But then, when the left crosser finishes crossing, they will release crossingL. Now left crossers can dominate the bridge as long as they continue to come one by one.

Suppose that two are crossing from the left using the approach from

above. Suppose that someone new comes from the left but no one is waiting on the right. Then, this left crosser, will not be able to acquire `crossingL`, they will fail in the attempt to acquire `waitR`, and then, they will try to acquire `crossingR`. They are stuck there forever.

Now, someone comes from the right, we have already put a left crosser waiting for `crossingR`. So the right crosser, will not be able to try-acquire `crossingR`, they will notice that someone is waiting on the left, and will also acquire on `crossingR`. They are also stuck there forever.

The bridge will continue to be open for crossers from the left (once those crossing finish crossing and exit).